# SE 423: Introduction to Mechatronics

## Lecture 5: DAC & ADC

Marius Juston

Wednesday, February 4th 2026

# Reminders

**Office Hours:**
- **Monday:** 4-6 PM**,** Neel
- **Tuesday:** 11-1 PM**,** Lakshmi
- **Tuesday:** 1-3 PM**,** Samuel
- **Tuesday:** 2-5 PM**,** Dan & Marius

**Lab:** Starting Lab 2 this week. This is also a 1-week lab!

**Homeworks:**
- Check-offs for homework 2 Ex 1 & 2 are due **February 17, 5PM (The end of office hours)**
- Check-offs for homework 2 Ex 3 & 4 are due **February 24, 5PM (The end of office hours)**
- Answers and code submissions for homework 2 are due **February 17, 9AM**

**LabVIEW (This week):**
- All check-offs for LabVIEW 1 are  due **February 5, 5PM**

**LabVIEW will be quickly checked-off at the beginning / end of your lab session.**

# Questions?

Homework, Lab, LabVIEW, quiz questions?

# Introduction

In the real world we have continuous signals:

- Temperature
- Sound
- Velocity
- Pressure
- Luminosity

Again, we have the CPU only able to process discrete binary values (0 or 1).

**ADC (Analog-to-Digital Converter):**
- "The Eyes/Ears." Converts physical signals into numbers the CPU can process.

**DAC (Digital-to-Analog Converter):**
- "The Hands/Voice." Converts processed numbers back into voltages to drive motors, speakers, etc.

# DAC

(Digital-to-Analog Converter)

# DAC

- A device that convert binary input number into a proportional analog voltage symbol.

$$V_{out} = V_{ref} \times \frac{X}{2^N}$$

Where $N$ represents the resolution, the larger $N$, the more possible subdivision values you can have (the more you have the more expensive it is!)

Where $X$ is the reference value you input to be converted.

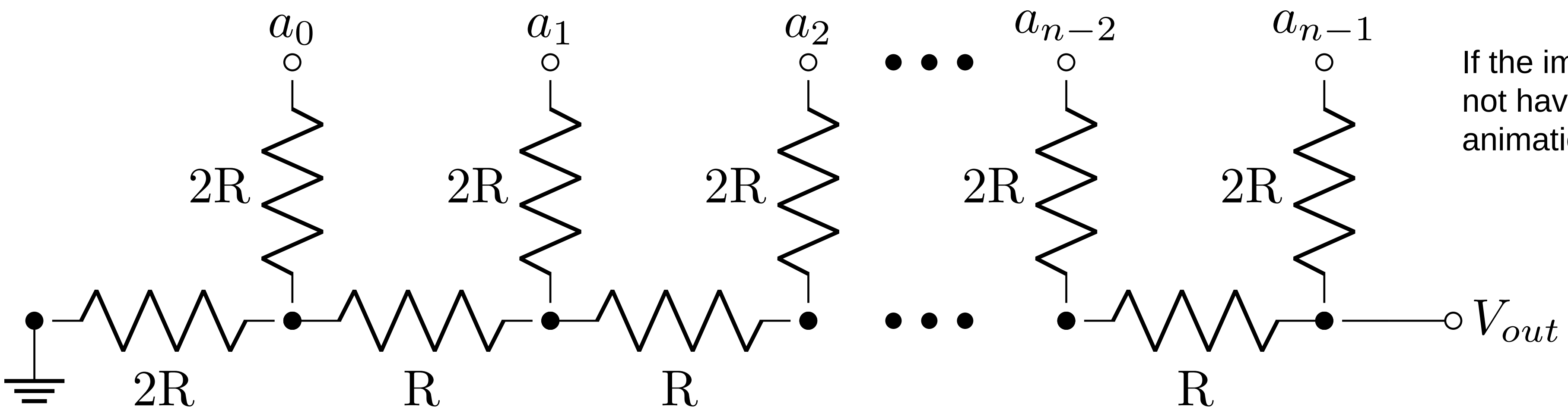
Specifically for our microcontroller we have,

$$\mathrm{DACOUT} = \frac{\mathrm{DACVALA} \times \mathrm{DACREF}}{4096}$$

How many bits of resolution do we have? **12**

# DAC – R-2R resistor ladder

A voltage mode resistor R-2R resistor ladder network.



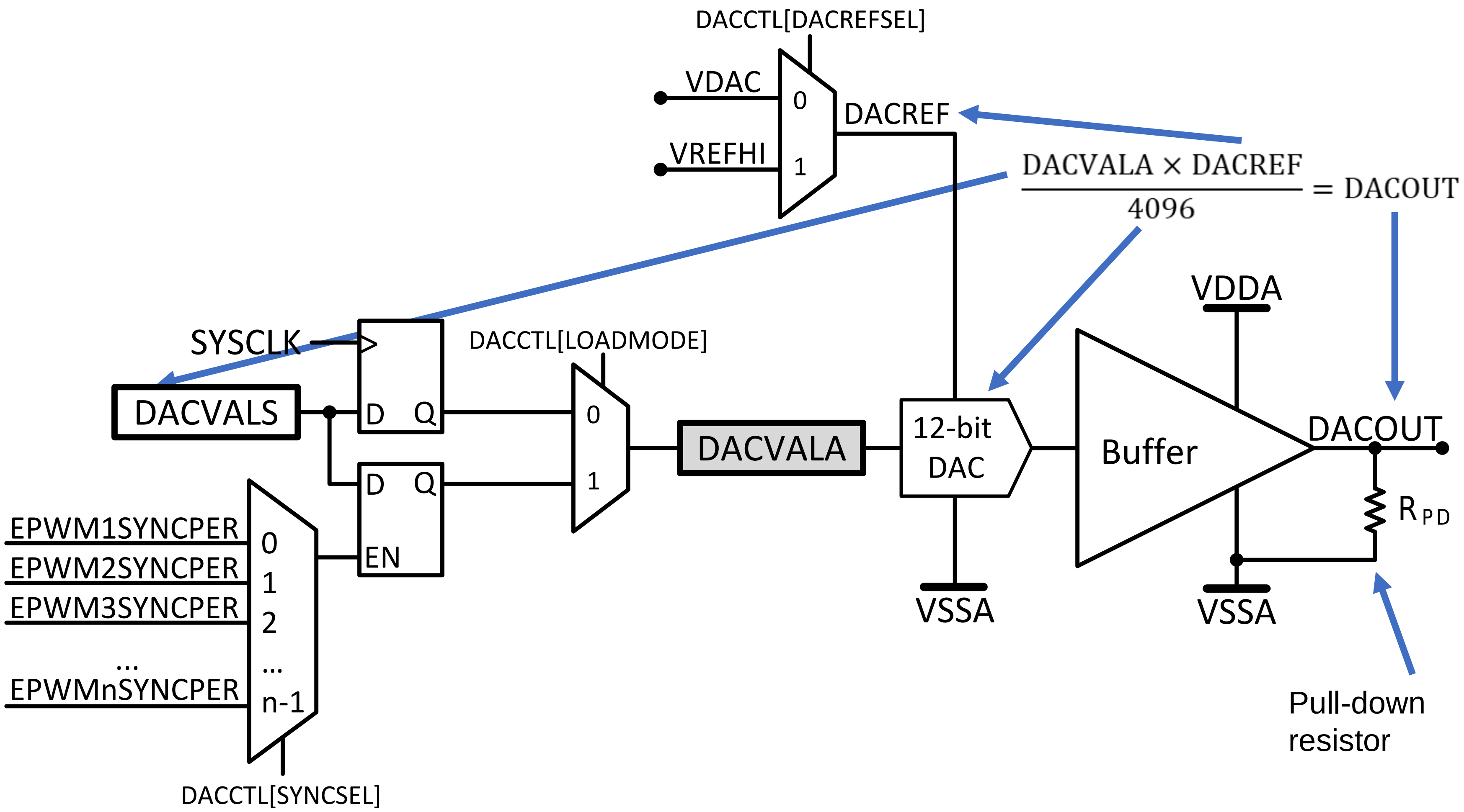

If the image is still here, I did not have time to make an animation

https://en.wikipedia.org/wiki/Resistor_ladder

It produces an analog output voltage from the digital integer $X$ composed of $n$ bits: $a_{n-1}$ (most significant bit, MSB) through bit $a_0$ (least significant bit, LSB).

# DAC – Buffered

**Figure 12-1. DAC Module Block Diagram**



# DAC

Each buffered DAC has the following features:

- 12-bit programmable internal DAC
- Selectable reference voltage source
  - Internal VDAC
  - External VREFHI
- Pull-down resistor on output
- Ability to synchronize with EPWMSYNCPER

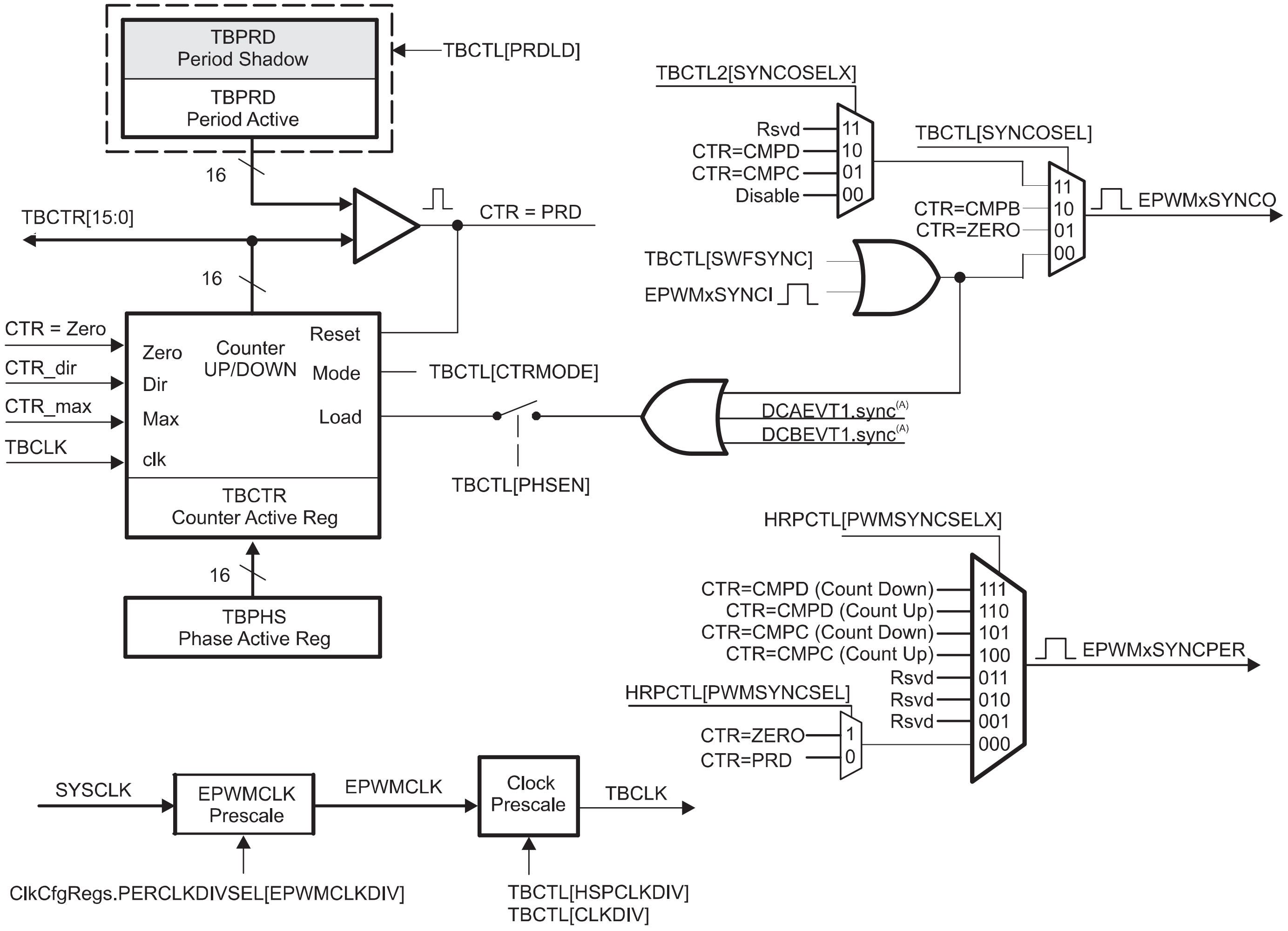
MCU DACs include an operational amplifier (Op-Amp) buffer at the output to drive loads (speakers, motor drivers) without dropping voltage due to impedance loading.

EPWMSYNCPER comes from the Time-Base submodule of the EPWM. (For a detailed description of how this signal is generated, refer to the Time-Base submodule chapter of the EPWM).

There are **3** DAC, DACA, DACB, DACC

# DAC – For later reference

**Figure 15-5. Time-Base Submodule Signals and Registers**




A. These signals are generated by the digital compare (DC) submodule.

# DAC

**Setup**

```
EALLOW;
DacaRegs.DACCTL.bit.DACREFSEL = 0;  // 0 from internal reference, 1 for external
DacaRegs.DACOUTEN.bit.DACOUTEN = 1; // DAC output enabled? (DAC-OUT-EN)
DacaRegs.DACVALS.all = 0;           // Clears current output value
DELAY_US(10);                       // Delay for buffered DAC to power up
EDIS;
```

**Setting**

```
DacaRegs.DACVALS.all = val;         // Sets DAC output (only keeps to first 12 bits)
```

# ADC

(Analog-to-Digital Converter)

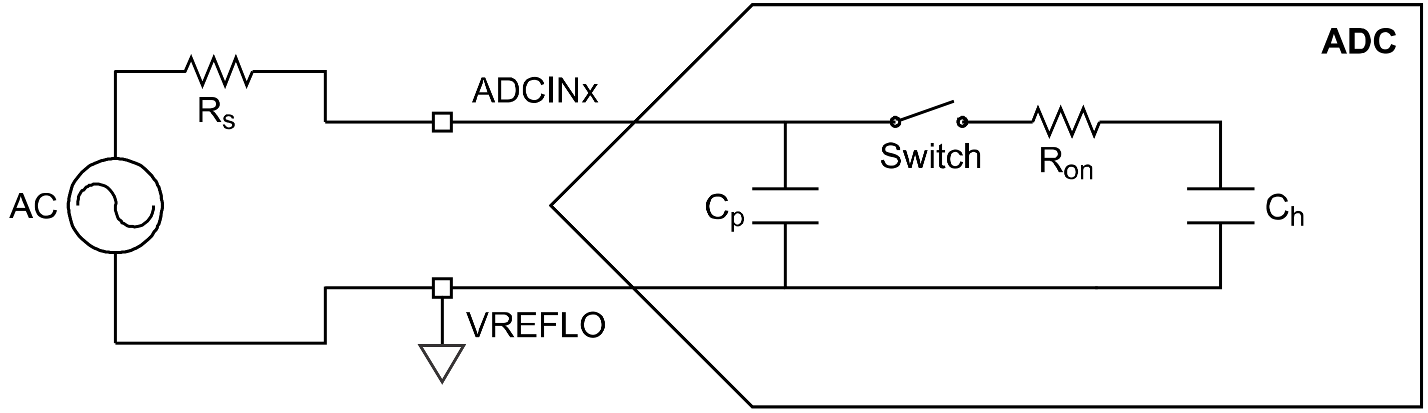
# ADC

Converts continuous voltage into discrete digital numbers.

3 step process:

- **Sample**: A switch closes, charging a tiny internal capacitor to match the input voltage
- **Holding**: The switch open, trapping the charge on the capacitor so it does not change during measurement
- **Quantizing**: The ADC circuitry measures the stored charge against a reference

**Figure 11-3. Single-Ended Input Model**



# ADC

Each ADC has the following features:

- Selectable resolution of 12 bits or 16 bits ( for the precision you want)
- Ratiometric external reference set by VREFHI and VREFLO pins
- Differential signal conversions (16-bit mode only)
- Single-ended signal conversions (12-bit mode only)
- Input multiplexer with up to 16 channels (single-ended) or 8 channels (differential)
- 16 configurable SOCs ( we will go through what this is a bit later)
- 16 individually addressable result registers

# ADC

Each ADC has the following features:

- Multiple trigger sources
  - S/W - software immediate start
  - All ePWMs - ADCSOC A or B
  - GPIO XINT2
  - CPU Timers 0/1/2 (from each C28x core present)
  - ADCINT1/2
- Four flexible PIE interrupts
- Burst mode
- Four post-processing blocks, each with:
  - Saturating offset calibration
  - Error from setpoint calculation
  - High, low, and zero-crossing compare, with interrupt and ePWM trip capability
  - Trigger-to-sample delay capture

# ADC

Not every channel may be pinned out from all ADCs. Check the datasheet for your device to determine which channels are available!

ADC and DAC LaunchPad Pins

| | | | | | | |
|---|---|---|---|---|---|---|
| 23/J3.3 | ADCIN14 | CMPIN4P | | J21.1 | ADCIND0 | COMIN7P |
| 24/J3.4 | ADCINC3 | CMPIN6N | | J21.3 | ADCIND1 | CMPIN7N |
| 25/J3.5 | ADCINB3 | CMPIN3N | | J21.5 | ADCIND2 | CMPIN8P |
| 26/J3.6 | ADCINA3 | CMPIN1N | Joystick Y | J21.7 | ADCIND3 | CMPIN8N |
| 27/J3.7 | ADCINC2 | CMPIN6P | | | | |
| 28/J3.8 | ADCINB2 | CMPIN3P | | Solder BNC | ADCIND4 | |
| 29/J3.9 | ADCINA2 | CMPIN1P | Joystick X | Solder BNC | ADCIND5 | |
| 30/J3.10 | ADCINA0 | DACA | | | | |
| 63/J7.3 | ADCIN15 | CMPIN4N | | | | |
| 64/J7.4 | ADCINC5 | CMPIN5N | | | | |
| 65/J7.5 | ADCINB5 | | | | | |
| 66/J7.6 | ADCINA5 | CMPIN2N | | | | |
| 67/J7.7 | ADCINC4 | CMPIN5P | | | | |
| 68/J7.8 | ADCINB4 | | Microphone | | | |
| 69/J7.9 | ADCINA4 | CMPIN2P | | | | |
| 70/J7.10 | ADCINA1 | DACB | | | | |

# ADC - Differential

The ADC supports two signal modes: **single-ended** and **differential**.

- In **single-ended** mode, the input voltage to the converter is sampled through a single pin (ADCINx), referenced to VREFLO.
- In **differential** signaling mode, the input voltage to the converter is sampled through a pair of input pins, one of which is the positive input (ADCINxP) and the other is the negative input (ADCINxN). The actual input voltage is the difference between the two (ADCINxP – ADCINxN).
- **Differential** signaling mode is advantageous because noise encountered on both inputs will be largely cancelled. Allowing for higher 16-bit precision.

**Figure 11-3. Single-Ended Input Model**

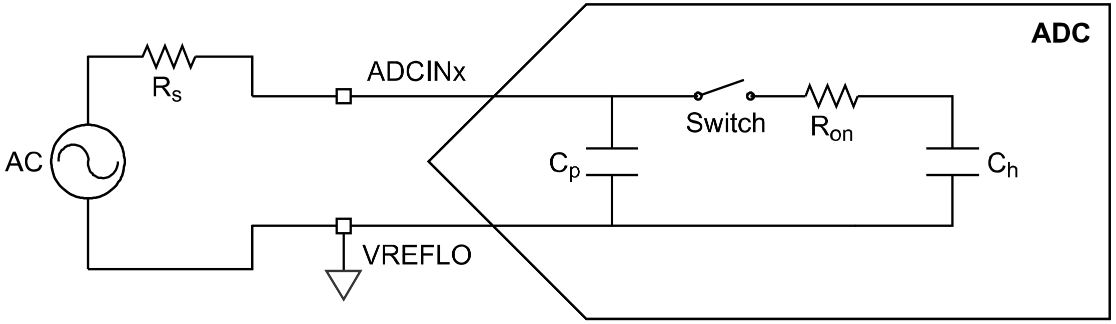
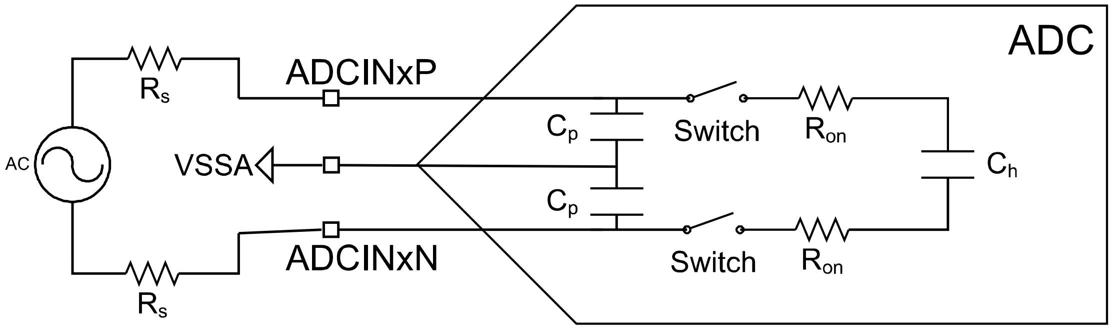



**Figure 11-4. Differential Input Model**



# ADC - Calculate

**Table 11-2. Analog to 12-bit Digital Formulas**

| | Analog Input | Digital Result |
|---|---|---|
| **Single-Ended** | when ADCINy ≤ VREFLO | ADCRESULTx = 0 |
| | when VREFLO < ADCINy < VREFHI | $\text{ADCRESULTx} = 4096\left(\frac{\text{ADCINy - VREFLO}}{\text{VREFHI - VREFLO}}\right)$ |
| | when ADCINy ≥ VREFHI | ADCRESULTx = 4095 |
| **Differential** | Invalid Mode | Invalid Mode |

**Table 11-3. Analog to 16-bit Digital Formulas**

| | Analog Input | Digital Result |
|---|---|---|
| **Single-Ended** | Invalid Mode | Invalid Mode |
| **Differential** | when ADCINyP - ADCINyN ≤ -VREFHI | ADCRESULTx = 0 |
| | when -VREFHI < ADCINyP - ADCINyN ≤ VREFHI | $\text{ADCRESULTx} = 65536\left(\frac{\text{ADCINyP - ADCINyN + VREFHI}}{\text{2 VREFHI}}\right)$ |
| | when ADCINyP - ADCINyN ≥ VREFHI | ADCRESULTx = 65535 |

# ADC – SAR (Successive Approximation Register)

- F28379D uses SAR. It works like a "binary search”. Convert continuous to digital.

For iteration $\mathrm{i}$ (where $\mathrm{i}$ goes from $\mathrm{N}\text{-}1$ down to $0$ ):

1. **Trial Step**: The SAR logic sets the current bit $\mathrm{b_i}$ to 1 (tentatively) and all lower bits to 0.
2. **DAC Output**: The internal DAC generates a trial voltage: $\mathrm{V_{trial}} = \mathrm{V_{accumulated}} + \frac{\mathrm{V_{ref}}}{2^{\mathrm{N\text{-}1\text{-}i}}} \times \frac{\mathrm{V_{ref}}}{2}$ *(Note: This simplifies to testing the half-way point of the remaining search interval).*
3. **Comparison**: The analog comparator checks the condition: $\mathrm{V_{in}} \geq \mathrm{V_{trial}}$
4. **Decision**:

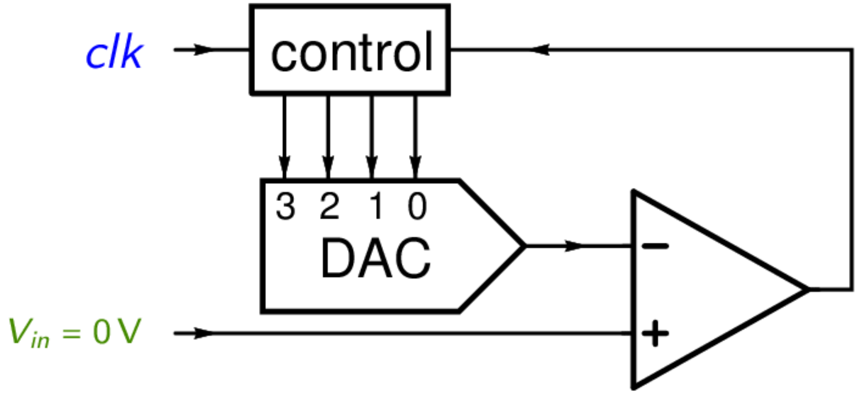
- If **True**: The comparator outputs '1'. The bit $\mathrm{b_i}$ remains 1. The trial voltage is kept as part of $\mathrm{V_{accumulated}}$
- If **False**: The comparator outputs '0'. The bit $\mathrm{b_i}$ is cleared to 0. The trial voltage is discarded.

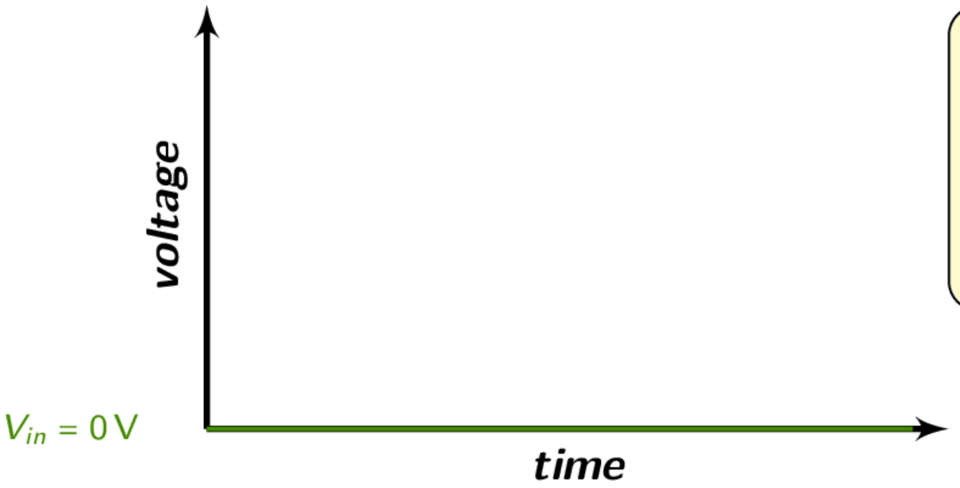
This process converges logarithmically (because it is binary search). The error at step $\mathrm{i}$ is strictly bounded by

$$\frac{\mathrm{V_{ref}}}{2^{\mathrm{N-i}}}$$

# ADC – SAR (Successive Approximation Register)

## Successive Approximation – example of a 4-bit ADC




Resolution:

| | |
|---|---|
| $5\,V \times 1/2$ | $2.5000\,V$ |
| $5\,V \times 1/4$ | $1.2500\,V$ |
| $5\,V \times 1/8$ | $0.6250\,V$ |
| $5\,V \times 1/16$ | $0.3125\,V$ |
| ... | |
| $5\,V \times 1/1024$ | $0.0049\,V$ |

https://en.wikipedia.org/wiki/Successive-approximation_ADC

# SOC

(Start-of-Conversion)

# SOC - Intro

In simple microcontrollers (like Arduino), you usually just say “Read analog value from pin A0 now!”. In more advanced controllers we use **SOCs**

SOCs is a configuration set that defines *what* happens, not just *when:*

You do not configure the ADC directly, you configure “SOC0” “SOC1”, etc…

Each SOC defines:

- **Trigger Source**: What causes the same? (Timer, Software, ePWM, GPIO).
- **Channel:** Which pin to sample? (e.g. ADCINA4).
- **Acquisition Window (ACQPS):** How long to hold the “sample” switch open?

# SOC – Trigger Sources

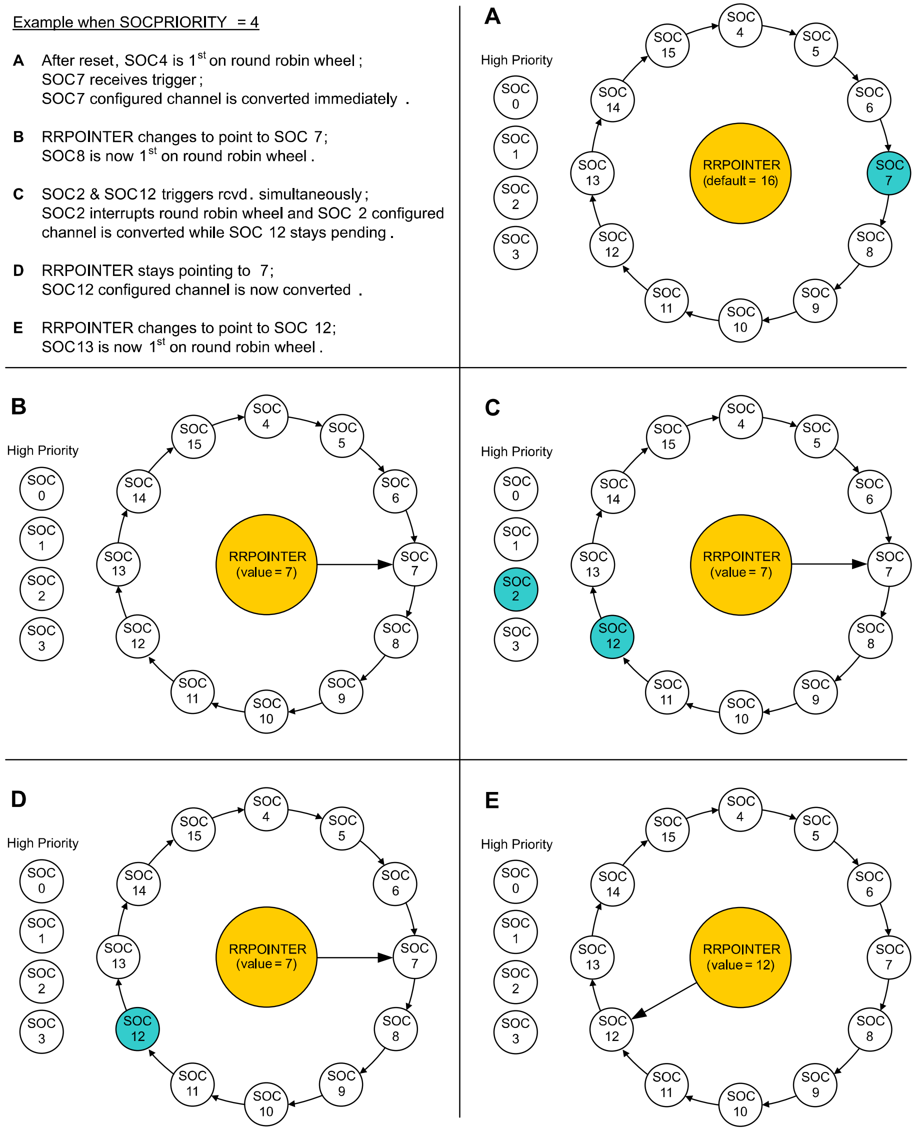
The ADC is an event-based machine.

For example, you can set SOC1 to trigger on **ePWM1** (for motor currents) and SOC1 to trigger on **CPU Timer 2** ( for temperature monitoring)

What to do if multiple triggers arrive at once? We use a **Round Robin**, to decide which SOC converts first. You can assign SOCs to have strict priority over others (critical safety loops)

# SOC – Trigger Sources

**Figure 11-6. High Priority Example**



# SOC – Acquisition Window

- The internal capacitor $C_h$ needs time to charge. If the source impedance is high (weak signal), it takes longer

```
AdcxRegs.ADCSOCxCTL.bit.ACQPS
```

ACQPS is a 9-bit register that can be set to a value between 0 and 511, resulting in an acquisition window duration of:

$$\text{Acquisition Window} = (\text{ACQPS} + 1) \times (\text{System Clock (SYSCLK) cycle time}).$$

The acquisition window duration is based on the System Clock (SYSCLK), not the ADC clock (ADCCLK)!

Why it matters: If you make this too short, the capacitor won't fully charge, and you will read the previous channel's "ghost" voltage (cross-talk).

# SOC – Interrupts

You don’t need an interrupt for *every* sample ( bit ).

You can set an interrupt to fire only after a *sequence* (e.g. After SOC3 completes)

**Early vs Late Interrupts**:

- **Early**: First the interrupt at the end of acquisition
- **Late**: At the end of conversion

# ADC – Single Conversion from ePWM

```
AdcaRegs.ADCSOC5CTL.bit.CHSEL = 1;    //SOC5 will convert ADCINA1
AdcaRegs.ADCSOC5CTL.bit.ACQPS = 19;   //SOC5 will use sample duration of 20 SYSCLK cycles
AdcaRegs.ADCSOC5CTL.bit.TRIGSEL = 10; //SOC5 will begin conversion on ePWM3 SOCB
```

# ADC – FFT Code (much more complicated example)

```
EALLOW;                                         //write configurations for all ADCs ADCB
AdcbRegs.ADCCTL2.bit.PRESCALE = 6;              //set ADCCLK divider to /4
AdcSetMode(ADC_ADCB, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration settings
//Set pulse positions to late
AdcbRegs.ADCCTL1.bit.INTPULSEPOS = 1;           //power up the ADCs
AdcbRegs.ADCCTL1.bit.ADCPWDNZ = 1;
DELAY_US(1000);                                 //delay for 1ms to allow ADC time to power up


AdcbRegs.ADCSOC0CTL.bit.CHSEL = 4;              //SOC0 will convert Channel you choose Does not have to be
B0
AdcbRegs.ADCSOC0CTL.bit.ACQPS = 99;             //sample window is acqps + 1 SYSCLK cycles = 500ns
AdcbRegs.ADCSOC0CTL.bit.TRIGSEL = 0x11;         // EPWM7 ADCSOCA or another trigger you choose will
trigger SOC0
AdcbRegs.ADCINTSEL1N2.bit.INT1SEL = 0;          //set to last SOC that is converted and it will set INT1
flag ADCB1
AdcbRegs.ADCINTSEL1N2.bit.INT1E = 1;            //enable INT1 flag, originally = 1 (need to set to 1 for
non-DMA)
AdcbRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;          //make sure INT1 flag is cleared
AdcbRegs.ADCINTSEL1N2.bit.INT1CONT = 1;         // Interrupt pulses regardless of flag state
EDIS;
```

# Questions?

The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN